

UNIT 1: FUNDAMENTALS OF CLOUD COMPUTING

Course Pattern: TE IT - 2024 Pattern (SPPU)
Status: Publication-Grade Theory Guide

Academic Term: Semester V (2026)
Course Code: PEC-321A-IT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 1.

Syllabus Topic	In-Depth Exam Coverage	Weightage / Priority
Foundations & Evolution	NIST Definition, Utility Computing model, Economic drivers (CapEx vs OpEx), Evolution path (Mainframe -> Distributed -> Grid -> Utility -> Cloud).	High Priority (8-10 Marks)
NIST Characteristics	5 Essential Characteristics (On-demand, Broad access, Pooling, Elasticity, Metered), Core Advantages, Limitations & Obstacles.	High Priority (5-8 Marks)
Cloud Service Models	IaaS, PaaS, SaaS architecture, tenant control boundaries, provider examples, Shared Responsibility Matrix.	Critical Priority (8-10 Marks)
Cloud Deployment Models	Public, Private (On-prem vs Hosted), Hybrid (Cloud Bursting), Community Cloud, Comprehensive Comparison Matrix.	Critical Priority (8-10 Marks)
Cloud Architecture	4-Layer Reference Model (Hardware, Hypervisor, Platform, Application), SOA principles, Web Services, REST APIs.	Medium Priority (6-8 Marks)
Storage & Data Management	Block, File, Object Storage (S3, Blob), Data Lifecycle, CAP Theorem & Consistency models, Cloud SQL vs NoSQL.	High Priority (8-10 Marks)
Cloud Service Providers	Hyperscalers (AWS, Azure, GCP services), Open-source infrastructure platforms (OpenStack, Eucalyptus, Cloud-Stack).	Medium Priority (4-6 Marks)

Foundations & Evolution of Cloud Computing

Definition of Cloud Computing

According to the **National Institute of Standards and Technology (NIST)**, Cloud Computing is defined as:

*“Cloud computing is a model for enabling convenient, **on-demand network access** to a **shared pool of configurable computing resources** (e.g., networks, servers, storage, applications, and services) that can be **rapidly provisioned and released** with **minimal management effort** or service provider interaction.”*

The core paradigm of cloud computing rests on **resource virtualization** and **utility computing**. Rather than purchasing, deploying, and maintaining physical data centers and physical servers, organizations rent compute capacity, storage blocks, and database instances on a metered pay-per-use basis from hyperscale vendors over the internet.

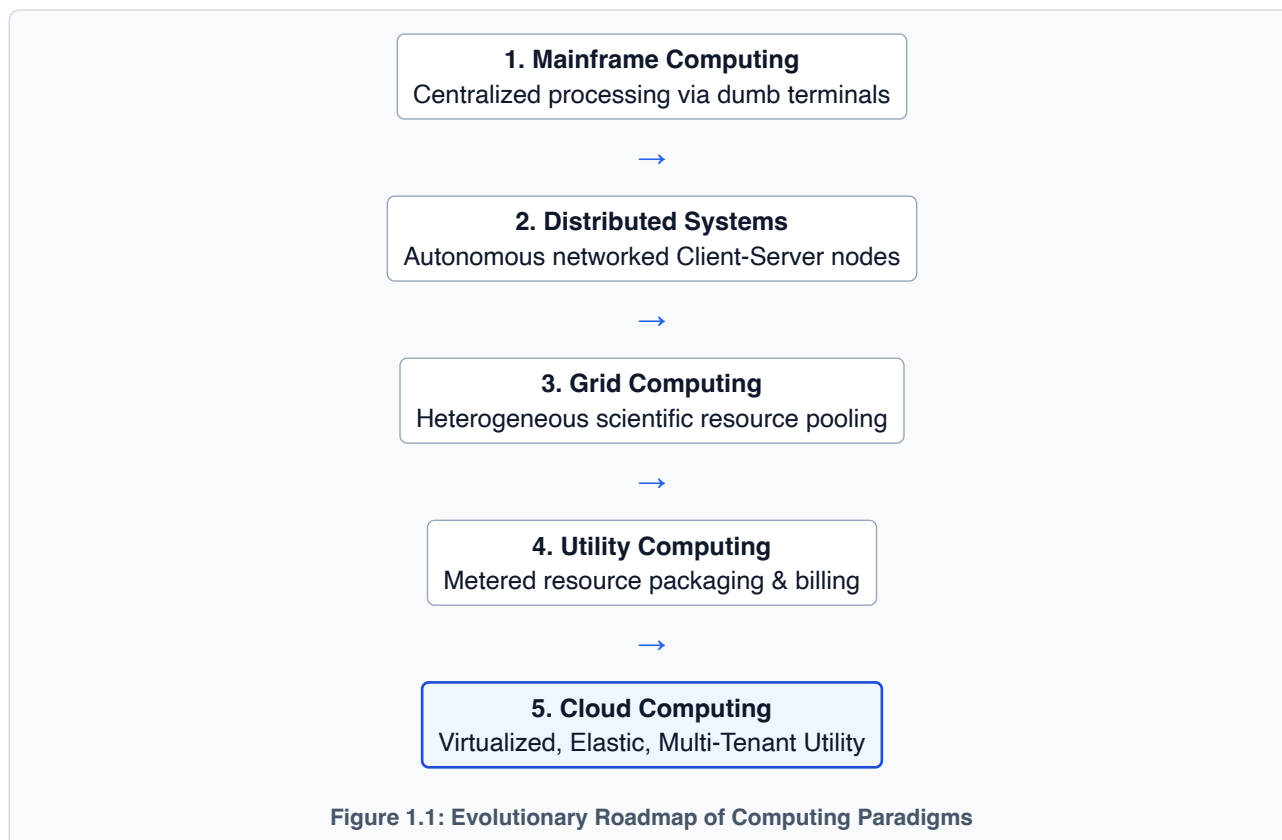
Economic Drivers & Operating Paradigm Shift

The widespread organizational transition from legacy on-premises IT infrastructure to cloud environments is driven by four primary financial and operational factors:

1. **Shift from Capital Expenditure (CapEx) to Operational Expenditure (OpEx):**
 - **Capital Expenditure (CapEx):** Traditional IT infrastructure demands massive upfront capital investments to acquire physical servers, network switches, SAN/NAS storage arrays, data center real estate, precision cooling systems, and perpetual software licenses.
 - **Operational Expenditure (OpEx):** Cloud computing replaces fixed upfront CapEx with dynamic, variable OpEx. Organizations incur costs only for the precise computational resources consumed during active execution (**Pay-as-you-Go / Metered Utility Pricing**).
2. **Agility and Speed of Innovation:**
 - Traditional physical infrastructure procurement and provisioning cycles span weeks or months.
 - Cloud environments allow software teams to spin up virtual instances, managed databases, and container clusters in seconds using self-service web dashboards or automated Infrastructure-as-Code (IaC) API calls.
3. **Elasticity & Scalability:**
 - Provisioning infrastructure to match dynamic traffic spikes dynamically without over-provisioning hardware that remains idle during off-peak periods.
4. **Offloading Maintenance Burden:**
 - Transfers lower-level facilities management, hardware maintenance, hypervisor patching, and physical security enforcement to cloud infrastructure providers.

Evolutionary Path of Cloud Computing

Cloud computing is the technological culmination of decades of advances in distributed systems, virtualization, networking, and utility billing models.



Paradigm	Core Concept & Architectural Model	Key Characteristics & Limitations
Mainframe Computing	Centralized processing on a single powerful master mainframe machine (e.g., IBM System/360) accessed by users via passive dumb terminals.	Extremely high reliability; massive hardware cost, zero elasticity, single point of failure.
Distributed Computing	Network of autonomous computers collaborating over network protocols to split compute tasks.	Improved concurrency and fault tolerance; complex middleware required for inter-process communication.
Grid Computing	Federation of geographically distributed, heterogeneous computing nodes pooled together to run massive scientific simulation workloads.	Handles compute-heavy tasks (e.g., CERN particle data processing); lacks dynamic user-level elasticity and self-service capabilities.
Utility Computing	Packaging compute cycles and storage allocations as a metered service analogous to public utilities (water, electricity).	Introduced pay-per-use billing models; direct precursor to cloud pricing structures.
Cloud Computing	Synthesis of hardware virtualization, utility pricing, SOA web services, and automated network provisioning.	On-demand self-service, rapid elasticity, broad network access, multi-tenancy, complete hardware abstraction.

Source: Srinivasan & Suresh, Ch 1, p. 3-14 | Source: Buyya et al., Ch 1, p. 3-12

NIST 5 Essential Characteristics, Advantages & Obstacles

NIST 5 Essential Characteristics

The official NIST model mandates five foundational characteristics that must be simultaneously satisfied to classify a technology platform as a genuine cloud service:

1. **On-Demand Self-Service:**
 - Consumers can provision computing capabilities—such as virtual CPU instances, network interfaces, and block storage—unilaterally and automatically without requiring manual intervention from service provider personnel.
2. **Broad Network Access:**
 - Cloud capabilities are accessible over standard network protocols (HTTP/HTTPS, SSH, REST APIs) using heterogeneous client platforms (thin clients, workstations, laptops, mobile devices).
3. **Resource Pooling:**
 - The provider's physical computing resources are pooled together to serve multiple consumers using a **multi-tenant model**, with physical and virtual resources dynamically assigned and reassigned according to demand. Customers generally have no control or exact knowledge over the exact physical location of hardware (location independence).
4. **Rapid Elasticity:**
 - Computational resources can be elastically provisioned and released—in many cases automatically based on auto-scaling rules—to scale rapidly outward during demand spikes and inward during quiet periods. To the end consumer, resources appear limitless.
5. **Measured Service:**
 - Cloud resource usage is automatically monitored, controlled, optimized, and reported using a metering capability appropriate to the service type (e.g., active storage gigabytes, bandwidth transferred, CPU execution hours). This provides transparency for both the provider and consumer.

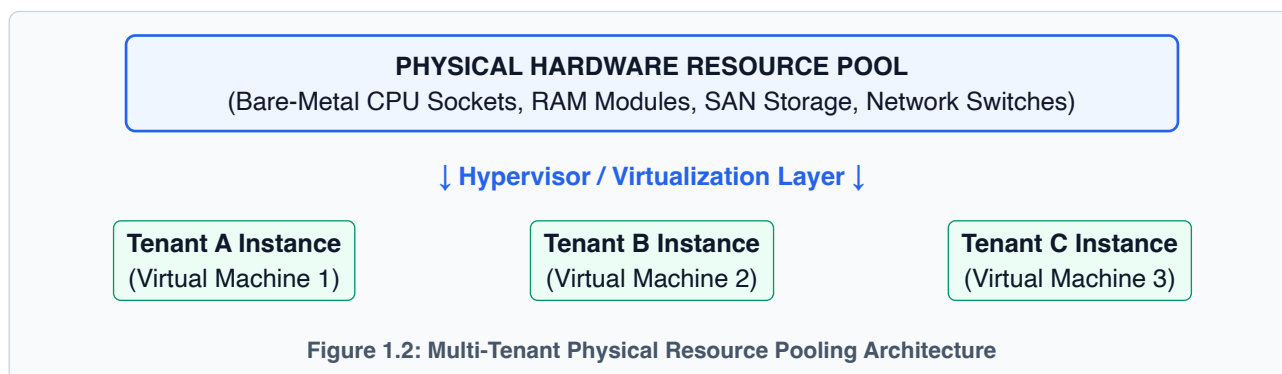


Figure 1.2: Multi-Tenant Physical Resource Pooling Architecture

Comprehensive Advantages of Cloud Computing

- **Capital Cost Optimization:** Converts massive fixed capital investments into operational pay-per-use expenses.
- **High Availability & Resiliency:** Hyperscalers deploy multi-datacenter infrastructure across independent **Availability Zones (AZs)** backed by strict Service Level Agreements (SLAs).
- **Automated Disaster Recovery:** Cloud native snapshots, cross-region asynchronous replication, and managed backups reduce Recovery Point Objective (RPO) and Recovery Time Objective (RTO).
- **Global Market Reach:** Global infrastructure deployments leverage edge locations and Content Delivery Networks (CDNs) to serve users with minimal latency.

Key Limitations and Adoption Obstacles

1. **Security & Multi-Tenant Privacy Risks:** Storing enterprise data on shared multi-tenant physical hardware exposes workloads to potential hypervisor escape vulnerabilities or side-channel leakage.

2. **Vendor Lock-In:** Proprietary cloud APIs, custom database formats (e.g., AWS DynamoDB), and specialized network constructs complicate cross-cloud migration.
3. **Network Latency Dependency:** Applications depend strictly on continuous high-speed network connectivity; WAN bandwidth bottlenecks directly impact response times.
4. **Data Sovereignty & Compliance:** Statutory regulations (e.g., GDPR, RBI data localization mandates, HIPAA) enforce strict boundaries on where customer data can be geographically stored.

Source: Sosinsky, Ch 1, p. 36-59 | Source: Srinivasan & Suresh, Ch 1, p. 9-12

Cloud Service Models (IaaS, PaaS, SaaS)

Cloud computing delivers services categorized into three primary architectural service models (SPI Model):

Infrastructure as a Service (IaaS)

Concept: IaaS delivers fundamental computing infrastructure—raw processing power, virtual machine instances, physical bare-metal servers, block/object storage volumes, and virtual network switches—over the network.

- **Core Characteristics:**
 - Provides hardware-level virtualization managed via hypervisors (KVM, VMware ESXi, Xen).
 - The customer retains complete control over operating systems (Linux/Windows), installed runtime environments, database engines, middleware, and software applications.
 - The customer manages virtual network topologies, subnets, routing tables, and security group firewall rules.
- **Provider Examples:** Amazon EC2, Microsoft Azure Virtual Machines, Google Compute Engine (GCE), DigitalOcean Droplets.

Platform as a Service (PaaS)

Concept: PaaS provides a managed development environment containing programming language runtimes, libraries, database engines, and software application frameworks required to build, test, and deploy applications without managing underlying servers.

- **Core Characteristics:**
 - Abstracts away hardware provisioning, operating system maintenance, hypervisor patching, web server configuration, and manual scaling.
 - Developers focus exclusively on application source code, data schema design, and runtime configurations.
 - Integrates automated application scaling, load balancing, managed database backends, and deployment pipelines.
- **Provider Examples:** AWS Elastic Beanstalk, Google App Engine, Heroku, Azure App Service, Red Hat OpenShift.

Software as a Service (SaaS)

Concept: SaaS delivers complete, fully managed end-user application software over the internet on a subscription or metered usage basis.

- **Core Characteristics:**
 - The application executes entirely on cloud provider infrastructure and is accessible via web browsers, mobile apps, or lightweight API clients.
 - The end consumer has no management or operational control over underlying servers, operating systems, network infrastructure, or application feature code.
 - Built on multi-tenant software architectures with automated background software updates and patch management.
- **Provider Examples:** Google Workspace (Gmail, Docs), Salesforce CRM, Microsoft 365, Slack, Zoom.

Shared Responsibility Matrix across Service Models

The **Shared Responsibility Model** dictates the security and management boundary between the cloud service provider and the customer across different architectural tiers:

Architectural Layer	On-Premises	IaaS	PaaS	SaaS
Applications	Customer	Customer	Customer	Provider
Data & Content	Customer	Customer	Customer	Customer/ Provider
Runtime Environment	Customer	Customer	Provider	Provider
Middleware	Customer	Customer	Provider	Provider
Operating System (OS)	Customer	Customer	Provider	Provider
Virtualization / Hypervisor	Customer	Provider	Provider	Provider
Compute Hardware	Customer	Provider	Provider	Provider
Storage Infrastructure	Customer	Provider	Provider	Provider
Networking & Data Center	Customer	Provider	Provider	Provider

Figure 1.3: Architectural Layer Ownership & Shared Responsibility Matrix

Source: Srinivasan & Suresh, Ch 16, p. 190-210 | Source: Sosinsky, Ch 4, p. 102-125

Cloud Deployment Models

Cloud deployment models define the environment location, administrative ownership, access permissions, and tenancy structure of the cloud infrastructure.

Public Cloud

- **Architecture:** Infrastructure is owned, operated, and maintained by a third-party cloud service provider (e.g., AWS, Azure) and delivered over the public internet to multiple organizations (multi-tenancy).
- **Characteristics & Trade-offs:** Zero upfront CapEx, near-infinite dynamic scalability, provider-managed hardware maintenance. However, physical infrastructure is shared across untrusted tenants.
- **Use Cases:** Public web portals, customer-facing applications, e-commerce backends, software development testing.

Private Cloud

- **Architecture:** Infrastructure is provisioned for exclusive use by a single organization (single-tenancy). It may exist on-premises (On-Premises Private Cloud) or be hosted by a third party (Hosted Private Cloud).
- **Characteristics & Trade-offs:** Grants maximum physical control, isolated data security, predictable performance, strict regulatory compliance. However, it incurs high upfront hardware CapEx and ongoing administrative maintenance costs.
- **Use Cases:** Banking transaction core systems, healthcare records databases, military defense workloads.

Hybrid Cloud

- **Architecture:** Combines public and private cloud environments connected by standardized technology (VPNs, dedicated fiber links) that enables data and workload portability.
- **Cloud Bursting Mechanism:** Workloads run primarily in a secure private cloud; when application demand exceeds internal compute capacity, the application automatically “bursts” excess demand into public cloud compute instances.
- **Characteristics & Trade-offs:** Balances strict security for sensitive data with elastic scalability for web frontends. Requires complex network routing, identity management, and orchestration.

Community Cloud

- **Architecture:** Infrastructure shared by multiple organizations that possess shared concerns, security policies, compliance mandates, or operational missions.
- **Characteristics & Trade-offs:** Spreads infrastructure capital costs across consortium members; provides higher security than public clouds but requires complex shared governance models.
- **Use Cases:** State government inter-agency portals, healthcare research consortiums, educational university networks.

Comparison Matrix of Cloud Deployment Models

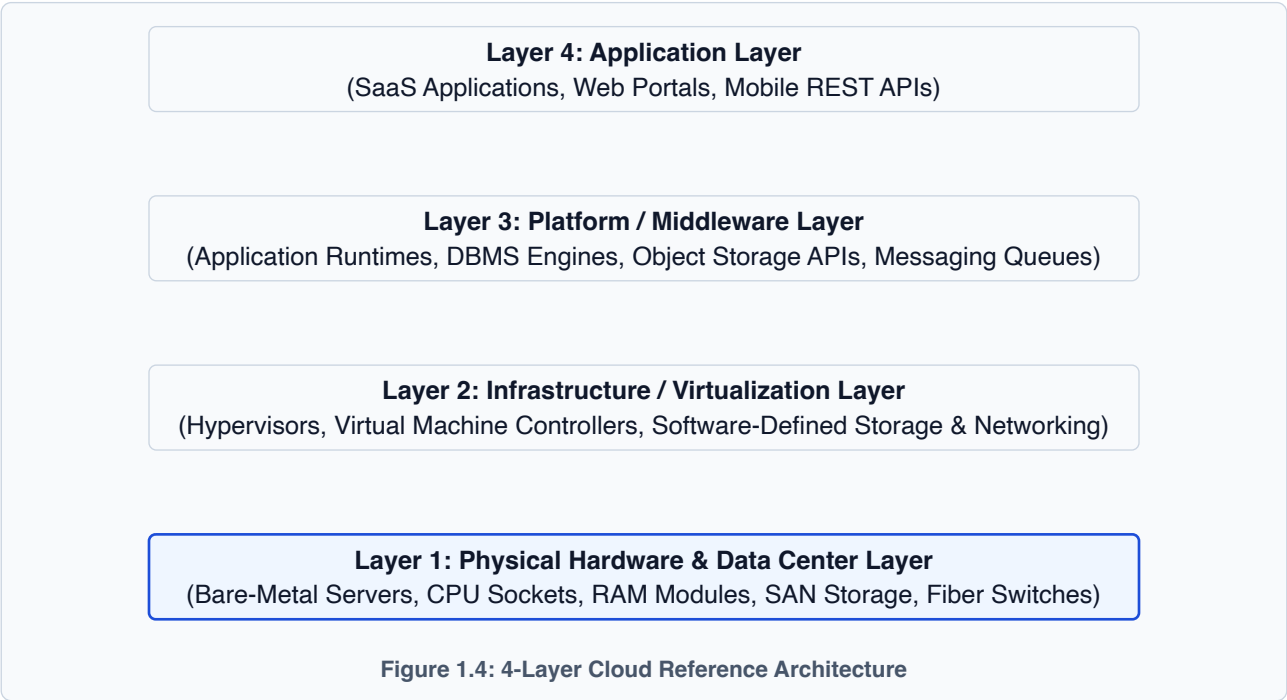
Parameter	Public Cloud	Private Cloud	Hybrid Cloud	Community Cloud
Tenancy Structure	Multi-tenant	Single-tenant	Mixed tenancy	Multi-organization
Cost Model	Low pay-per-use OpEx	High CapEx & OpEx	Balanced OpEx/ CapEx	Shared cost pool
Security & Privacy	Standard / Shared	Maximum isolation	Granular / Flexible	Domain-specific
Scalability	Near-infinite	Hardware bounded	High (Cloud Bursting)	Moderate
Governance	Provider managed	In-house / Customer	Split governance	Joint consortium

Source: Srinivasan & Suresh, Ch 3, p. 37-44 | Source: Buyya et al., Ch 4, p. 125-130

Cloud Architecture & Component Stack

4-Layer Cloud Reference Architecture

A standard cloud architectural framework comprises four operational layers:



Virtualization & Hypervisor Architecture

Virtualization is the foundational technology that enables cloud multi-tenancy by creating virtual representations of physical compute, storage, and networking resources.

The **Hypervisor (Virtual Machine Monitor - VMM)** is the software layer that intercepts system hardware requests from Guest Virtual Machines (VMs) and manages physical CPU, memory, and I/O allocation.

Parameter	Type-1 Hypervisor (Bare-Metal)	Type-2 Hypervisor (Hosted)
Architectural Placement	Runs directly on bare-metal physical hardware without an underlying host OS.	Executes as a software application on top of an existing Host Operating System.
Performance & Overhead	High efficiency, low virtualization overhead, near-native execution speeds.	Higher latency and resource overhead due to host OS context switching.
Security Isolation	High isolation; failure of one VM does not impact others.	Vulnerable to underlying Host OS security flaws and crashes.
Enterprise Examples	VMware ESXi, KVM (Kernel-based VM), Xen, Microsoft Hyper-V.	VMware Workstation, Oracle VirtualBox, Parallels Desktop.

Service-Oriented Architecture (SOA) & REST Web Services

Cloud infrastructure components communicate using principles of **Service-Oriented Architecture (SOA)**:

- **Loose Coupling:** Components interact through formal interface contracts without exposing underlying implementation logic.
- **RESTful Web Services:** Cloud resources are assigned unique Uniform Resource Identifiers (URIs) and manipulated statelessly using standard HTTP verbs (GET, POST, PUT, DELETE).

Source: Thomas Erl et al., Ch 4, p. 111-130

Cloud Storage & Data Management

Cloud Storage Models

Cloud platforms provide three primary data storage abstractions:

Storage Model	Architecture & Characteristics	Examples & Typical Usage
Block Storage	Data is split into raw, fixed-size blocks without metadata; attached directly to virtual machines as virtual hard drives. Provides high IOPS and low latency.	AWS Elastic Block Store (EBS), Azure Managed Disks. Used for OS boot volumes, relational databases (MySQL, PostgreSQL).
File Storage	Hierarchical file and folder tree structure shared concurrently across multiple virtual instances using network protocols (NFS, SMB).	AWS Elastic File System (EFS), Azure Files. Used for shared application content repositories, media pipelines.
Object Storage	Flat namespace storing data as discrete objects (Data payload + Custom Metadata + Globally Unique ID). Accessed via HTTP REST APIs.	Amazon S3, Google Cloud Storage, Azure Blob. Used for unstructured media, backups, data lake archives.

CAP Theorem in Distributed Cloud Storage

Distributed cloud storage systems replicated across multiple geographical data centers are subject to the trade-offs defined by the **CAP Theorem**:

CAP Theorem Rule: A distributed data storage system can simultaneously provide at most **two** of the following three guarantees during a network partition:

1. **Consistency (C):** Every read operation receives the most recent write or an error.
2. **Availability (A):** Every non-failing request receives a non-error response without guarantee that it contains the most recent write.
3. **Partition Tolerance (P):** The system continues operating despite arbitrary network packet drops or node network partitions.

- **Consistency Models:**

- **Strong Consistency:** Guarantees that all node replicas immediately reflect new data following a write operation (higher latency).
- **Eventual Consistency:** Replicas synchronize asynchronously over time. Offers lower write latency and high availability; reads may return temporarily stale data.

- **Cloud SQL vs NoSQL Databases:**

- **Cloud RDBMS (SQL):** Managed relational engines (AWS RDS, Azure SQL) enforcing **ACID** properties (Atomicity, Consistency, Isolation, Durability) via vertical hardware scaling (**Scale-Up**).
- **Cloud NoSQL:** Distributed databases (AWS DynamoDB, MongoDB Atlas) enforcing **BASE** properties (Basically Available, Soft-state, Eventual consistency) via horizontal node sharding (**Scale-Out**).

Overview of Major Cloud Service Providers

Hyperscale Cloud Providers

Amazon Web Services (AWS)

- **Compute:** EC2, Lambda
- **Storage:** S3, EBS, EFS
- **Database:** RDS, DynamoDB, Aurora

Microsoft Azure

- **Compute:** Azure VMs, App Service
- **Storage:** Azure Blob, Managed Disks
- **Database:** Azure SQL, Cosmos DB

Google Cloud Platform (GCP)

- **Compute:** Compute Engine, GKE
- **Storage:** Cloud Storage
- **Database:** Cloud Bigtable, Spanner

Open-Source Private Cloud Frameworks

1. **OpenStack:** Modular open-source cloud operating system controlling data center compute pools (Nova), block storage (Cinder), object storage (Swift), and networking (Neutron) via a unified dashboard (Horizon).
2. **Eucalyptus:** Open-source private cloud platform providing API compatibility with AWS EC2 and S3 interfaces.
3. **CloudStack:** Turnkey open-source hypervisor-agnostic IaaS orchestration engine.

Source: Srinivasan & Suresh, Ch 14, p. 171-180